

# Verification Plan Report

## Project: AXI-Stream Master VIP Verification Plan

Version: 1.0

Verification Role: Master

### 1. UVM Environment Architecture

```
graph TD
    classDef test_layer fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px,color:#7b1fa2;
    classDef test_obj fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px,stroke-dasharray: 5
5,color:#7b1fa2;
    classDef env_layer fill:#e3f2fd,stroke:#1565c0,stroke-width:2px,color:#1565c0;
    classDef env_obj fill:#e3f2fd,stroke:#1565c0,stroke-width:2px,stroke-dasharray: 5
5,color:#1565c0;
    classDef agent_layer fill:#e0f2f1,stroke:#00695c,stroke-width:2px,color:#00695c;
    classDef agent_obj fill:#e0f2f1,stroke:#00695c,stroke-width:2px,stroke-dasharray: 5
5,color:#00695c;
    classDef dut_layer fill:#fff8e1,stroke:#f57f17,stroke-width:2px,color:#f57f17;

    subgraph UVM_Test_Layer [axi_stream_test]
        SeqItem[axi_stream_seq_item]:::test_obj
        Seq[axi_stream_master_sequence]:::test_obj
        subgraph Env_Layer [axi_stream_env]
            Scoreboard[axi_stream_scoreboard]:::env_layer
            Env_Cfg[axi_stream_env_config]:::env_obj
            subgraph Agent_Layer [Master Agent (VIP)]
                Sequencer[axi_stream_master_sequencer]:::agent_layer
                Agent_Cfg[axi_stream_master_agent_config]:::agent_obj
                Driver[axi_stream_master_driver]:::agent_layer
                Monitor[axi_stream_master_monitor]:::agent_layer
                Sequencer --> Driver
                Agent_Cfg --> Sequencer
            end
        end
    end

    subgraph Top_Level [Hardware Top]
        VIF[Virtual Interface (axi_stream_if)]:::dut_layer
        DUT[DUT Slave Receiver]:::dut_layer
        VIF <--> DUT
    end

    Driver --- VIF
    Monitor --- VIF
    Monitor --> Scoreboard
```

# Verification Plan Report

## 2. Test Requirements

| Index      | Feature/Sub                                 | Name                                   | Description                                                                                                                                                                                                                                                                                                                       |
|------------|---------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| REQ_MST_01 | Handshake Protocol<br>(Basic Transfer)      | TVALID/TREADY Handshake Conformance    | The Master must assert TVALID to indicate valid data. A transfer only occurs when both TVALID and TREADY are HIGH at the rising edge of ACLK.<br><br>Verification strategy involves testing all three handshake scenarios: TVALID before TREADY, TREADY before TVALID, and simultaneous assertion.                                |
| REQ_MST_02 | Handshake Protocol<br>(Master Stability)    | TVALID Stability during Handshake      | Once TVALID is asserted, it must remain HIGH and all associated signals (TDATA, TKEEP, TSTRB, TLAST, TID, TDEST, TUSER) must stay unchanged until the handshake completes.<br><br>Verification intent is to inject TREADY stalls and verify that the Master Driver does not prematurely toggle TVALID or change payload contents. |
| REQ_MST_03 | Handshake Protocol<br>(Deadlock Prevention) | Master TVALID Independence             | The Master Transmitter must NOT wait for TREADY to be asserted before asserting TVALID.<br><br>Verification strategy includes holding TREADY low indefinitely and ensuring the Master VIP still asserts TVALID when data is ready in its internal queue.                                                                          |
| REQ_MST_04 | Packetization<br>(Frame Boundary)           | TLAST Framing Logic                    | TLAST indicates the boundary of a packet/frame. Its assertion must occur only on the last byte transfer of a grouping.<br><br>Verification strategy involves randomized packet lengths and verifying TLAST assertion at the exact designated cycle relative to TDATA byte counts.                                                 |
| REQ_MST_05 | Data Integrity<br>(Optional Sidebands)      | Data Qualifiers (TKEEP/TSTRB) Matching | TKEEP and TSTRB qualify bytes within TDATA. If absent, they default to all-HIGH.<br><br>Verification involves testing null-byte insertions where TKEEP=0 and checking if the Scoreboard correctly masks these bytes in comparison.                                                                                                |

## 3. Test Cases & Mapping

| Index      | Scenario                                                                                        | TR Mapping |
|------------|-------------------------------------------------------------------------------------------------|------------|
| TC_MST_001 | Directed Handshake: TVALID asserted 3 cycles before TREADY.                                     | REQ_MST_01 |
| TC_MST_002 | Directed Handshake: TREADY asserted 2 cycles before TVALID.                                     | REQ_MST_01 |
| TC_MST_003 | Directed Handshake: Simultaneous assertion of TVALID and TREADY.                                | REQ_MST_01 |
| TC_MST_004 | Stability Check: Assert TVALID, then stall TREADY for 10 cycles. Verify TDATA remains constant. | REQ_MST_02 |
| TC_MST_005 | Deadlock Check: Ensure TVALID asserts even when TREADY is tied LOW.                             | REQ_MST_03 |

# Verification Plan Report

|            |                                                                                |                        |
|------------|--------------------------------------------------------------------------------|------------------------|
| TC_MST_006 | Packet Burst: Send random-sized packets (1 to 64 bytes) with TLAST at the end. | REQ_MST_04             |
| TC_MST_007 | Zero Data Transfer: Handshake with TLAST=1 but all TKEEP bits=0.               | REQ_MST_04, REQ_MST_05 |

## 4. Functional Coverage Groups

| Group Name     | Description                                                |
|----------------|------------------------------------------------------------|
| cg_handshake   | Cross coverage of TVALID and TREADY transitions.           |
| cg_tlast_burst | Cover points for various packet lengths ending with TLAST. |

## 5. Interface Signals

| Signal Name | Direction | Width | Description                               |
|-------------|-----------|-------|-------------------------------------------|
| ACLK        | Input     | 1     | Global clock signal.                      |
| ARESETn     | Input     | 1     | Global reset signal (active LOW).         |
| TVALID      | Output    | 1     | Master indicates a valid transfer.        |
| TREADY      | Input     | 1     | Slave indicates readiness to accept.      |
| TDATA       | Output    | 32    | Data Payload (configurable width).        |
| TKEEP       | Output    | 4     | Byte-qualifier for data and merged bytes. |
| TLAST       | Output    | 1     | Indicates end of packet.                  |

## 6. Verification Environment Structure

```
axi_stream_master_vip_tb/  
|-- master_agent/  
|   |-- axi_stream_master_driver.sv  
|   |-- axi_stream_master_monitor.sv  
|   |-- axi_stream_master_sequencer.sv  
|   |-- axi_stream_master_agent_config.sv  
|-- env/  
|   |-- axi_stream_scoreboard.sv  
|   |-- axi_stream_env_config.sv  
|-- sequences/  
|   |-- axi_stream_master_base_sequence.sv  
|   |-- axi_stream_seq_item.sv  
|-- top/  
|   |-- axi_stream_master_tb_top.sv  
|   |-- axi_stream_master_test.sv  
|   |-- axi_stream_defines.sv
```

# Verification Plan Report

## 7. SystemVerilog Implementation

### axi\_stream\_master\_driver.sv (Architecture Blueprint)

```
class axi_stream_master_driver extends uvm_driver #(axi_stream_seq_item);
    virtual axi_stream_if vif;

    task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(req);
            drive_transfer(req);
            seq_item_port.item_done();
        end
    endtask

    task drive_transfer(axi_stream_seq_item item);
        vif.TDATA  <= item.tdata;
        vif.TKEEP  <= item.tkeep;
        vif.TLAST  <= item.tlast;
        vif.TVALID <= 1'b1;

        @(posedge vif.ACLK);
        while(!vif.TREADY) @(posedge vif.ACLK); // wait for handshake

        vif.TVALID <= 1'b0; // End of transfer
    endtask
endclass
```